Experiment No.	03
Title	**Write a Program using JavaScript (Array)**
Course Outcome	CO1

1. Objective / Aim

To write a JavaScript program that demonstrates array creation and the commonly used built-in array methods — element access, insertion, deletion, searching, iteration, sorting, reversing, and joining.

2. System Requirements

Hardware

Processor	Intel Core i5 or equivalent
RAM	8 GB or above
Storage	256 GB SSD / HDD

Software

Operating System	Windows 11 Home
Code Editor	Visual Studio Code
Runtime	Node.js (JavaScript execution)
Terminal	VS Code Integrated Terminal

3. Architecture / Design

File Structure

```
folder structure
lab3_exp3/
  └── lab3_1/
        └── exp3_1.js    <-- JavaScript array program
```

Program Logic

- Declare an array of fruit names.
- Access elements using index (first, last via length - 1).
- Add elements — push() (end), unshift() (beginning).
- Remove elements — pop() (end), shift() (beginning).
- Read array length using the .length property.
- Search elements using indexOf() and includes().
- Iterate and display all elements using a for loop.
- Sort elements alphabetically using sort().
- Reverse the array order using reverse().
- Convert the array to a comma-separated string using join().

4. Source Code — exp3_1.js

exp3_1.js

```
let fruits = ["Apple", "Banana", "Mango", "Orange", "Grapes"];

console.log("Original Array:", fruits);

// Access
console.log("First Fruit:", fruits[0]);
console.log("Last Fruit:", fruits[fruits.length - 1]);

// Add
fruits.push("Pineapple");
console.log("After Push:", fruits);

// Remove
fruits.pop();
console.log("After Pop:", fruits);

// Add at beginning
fruits.unshift("Watermelon");
console.log("After Unshift:", fruits);

// Remove from beginning
fruits.shift();
console.log("After Shift:", fruits);

// Length
console.log("Number of Fruits:", fruits.length);

// Search
console.log("Index of Mango:", fruits.indexOf("Mango"));
console.log("Is Apple Available:", fruits.includes("Apple"));

// Display using loop
console.log("All Fruits:");
for (let i = 0; i < fruits.length; i++) {
    console.log(fruits[i]);
}

// Sort
fruits.sort();
console.log("Sorted Fruits:", fruits);

// Reverse
fruits.reverse();
```

```
console.log("Reverse Fruits:", fruits);

// Join
console.log("Fruit List:", fruits.join(", "));
```

5. Compilation / Execution Steps

Step 1 Create a project folder named lab3_exp3/, and inside it a subfolder lab3_1/.

Step 2 Create exp3_1.js inside lab3_1/ and paste the JavaScript source code.

Step 3 Open the lab3_exp3/ folder in Visual Studio Code.

Step 4 Open the integrated terminal (Ctrl + `) in VS Code.

Step 5 Navigate to the lab3_1/ folder: cd lab3_1

Step 6 Run the program using Node.js: node exp3_1.js

Step 7 Verify the console output shows the result of every array operation in sequence.

6. Expected Output

The output below is the actual console result produced by running exp3_1.js with Node.js, showing every array operation — access, push/pop, unshift/shift, search, loop, sort, reverse, and join — in order.

Terminal Output

```
Original Array: [ 'Apple', 'Banana', 'Mango', 'Orange', 'Grapes' ]
First Fruit: Apple
Last Fruit: Grapes
After Push: [ 'Apple', 'Banana', 'Mango', 'Orange', 'Grapes', 'Pineapple' ]
After Pop: [ 'Apple', 'Banana', 'Mango', 'Orange', 'Grapes' ]
After Unshift: [ 'Watermelon', 'Apple', 'Banana', 'Mango', 'Orange',
'Grapes' ]
After Shift: [ 'Apple', 'Banana', 'Mango', 'Orange', 'Grapes' ]
Number of Fruits: 5
Index of Mango: 2
Is Apple Available: true
All Fruits:
Apple
Banana
Mango
Orange
Grapes
Sorted Fruits: [ 'Apple', 'Banana', 'Grapes', 'Mango', 'Orange' ]
Reverse Fruits: [ 'Orange', 'Mango', 'Grapes', 'Banana', 'Apple' ]
Fruit List: Orange, Mango, Grapes, Banana, Apple
```

7. Result / Conclusion

The experiment was successfully completed. A JavaScript program was written and executed to demonstrate array operations — element access, addition (push/unshift), removal (pop/shift), length retrieval, searching (indexOf/includes), iteration using a for loop, sorting, reversing, and joining. The program was run using Node.js, and the console output confirmed correct behaviour of every built-in array method used, fulfilling Course Outcome CO1.

Student Signature

Faculty / Lab In-charge Signature